



**Министерство науки и высшего образования Российской
Федерации**
**Федеральное государственное бюджетное образовательное
учреждение высшего образования**
**«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

**Лабораторная работа № 2
по дисциплине «Анализ Алгоритмов»**

Тема Умножение матриц

Студент Поляков А.И.

Группа ИУ7-52Б

Преподаватели Волкова Л.Л., Строганов Д.М.

Москва, 2024

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 Аналитическая часть	6
1.1 Стандартный алгоритм умножения матриц	6
1.2 Алгоритм Винограда	6
1.3 Модель вычислений	7
1.4 Вывод	8
2 Конструкторская часть	9
2.1 Требования к программному обеспечению	9
2.2 Разработка алгоритмов	9
2.3 Оценка трудоемкости	12
2.4 Вывод	14
3 Технологическая часть	15
3.1 Средства разработки	15
3.2 Реализация алгоритмов	15
3.3 Функциональные тесты	18
3.4 Вывод	18
4 Исследовательская часть	19
4.1 Технические характеристики	19
4.2 Временные характеристики	19
4.3 Вывод	20
ЗАКЛЮЧЕНИЕ	21

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	22
---	-----------

ВВЕДЕНИЕ

Целью данной лабораторной работы является освоение метода оценки трудоемкости алгоритмов. В рамках работы рассматриваются стандартный алгоритм умножения матриц и алгоритм Винограда. Первый представляет собой классический метод, а алгоритм Винограда — оптимизированная версия, снижающая количество операций.

Задачи работы:

- Описание алгоритмов умножения матриц — стандартного и Винограда.
- Оценка трудоемкости алгоритмов по листингам, включая вычислительные операции (л.с.) и хранимые состояния (х.с.) для алгоритма Винограда и его оптимизированной версии.
- Разработка программного обеспечения
- Проведение замеров процессорного времени выполнения алгоритмов при изменении линейного размера матриц.
- Сравнительный анализ на основе графиков зависимости времени выполнения от размера матриц для исходной и оптимизированной версий алгоритма Винограда.

1 Аналитическая часть

В данной лабораторной работе рассматриваются два алгоритма умножения матриц: **стандартный алгоритм** и **алгоритм Винограда**.

1.1 Стандартный алгоритм умножения матриц

Стандартный алгоритм умножения матриц используется в линейной алгебре и является базовым методом для перемножения двух матриц. Пусть даны две матрицы A размером $n \times m$ и B размером $m \times k$, тогда результатом умножения будет матрица C размером $n \times k$, где каждый элемент c_{ij} вычисляется по следующей формуле:

$$c_{ij} = \sum_{r=1}^m a_{ir} \cdot b_{rj} \quad (1.1)$$

Общая сложность данного алгоритма составляет $O(n \cdot m \cdot k)$, что делает его неэффективным для больших матриц. В типичном случае при $n = m = k$ его сложность оценивается как $O(n^3)$, что может быть ограничивающим фактором при обработке больших данных. [3]

1.2 Алгоритм Винограда

Алгоритм Винограда представляет собой улучшенную версию стандартного алгоритма умножения матриц, который позволяет уменьшить количество операций умножения за счёт предварительного вычисления вспомогательных величин. Пусть даны матрицы A и B , как и ранее. Для матрицы A предварительно вычисляются суммы для каждой строки, а для матрицы B — для каждого столбца:

$$\text{row_factor}[i] = \sum_{r=1, r \text{ нечет}}^{m-1} a_{i,r} \cdot a_{i,r+1} \quad (1.2)$$

$$\text{col_factor}[j] = \sum_{r=1, r \text{ нечет}}^{m-1} b_{r,j} \cdot b_{r+1,j} \quad (1.3)$$

Затем элементы матрицы C вычисляются по формуле:

$$c_{ij} = -\text{row_factor}[i] - \text{col_factor}[j] + \sum_{r=1, r \text{ нечет}}^{m-1} (a_{ir} + b_{r+1,j}) \cdot (a_{ir+1} + b_{r,j}) \quad (1.4)$$

Если m нечётно, добавляется ещё одно произведение для последнего элемента.

Таким образом, алгоритм Винограда снижает количество операций умножения за счёт предварительных вычислений и замены некоторых умножений сложениями. Это делает его более эффективным для определённых типов задач, особенно когда размер матриц велик.

Однако на практике эффективность алгоритма Винограда может быть снижена за счёт увеличения числа операций сложения и вычитания, а также дополнительных затрат на память для хранения промежуточных данных. Его сложность в среднем остаётся $O(n \cdot m \cdot k)$, но константы уменьшаются по сравнению со стандартным алгоритмом. [4]

1.3 Модель вычислений

В данной работе будет выполняться оценка трудоёмкости алгоритмов, поэтому будет введена модель вычислений:

Трудоёмкость базовых операций

- Единичные операции: $=, ==, -, +, <, <=, []$
- Операции трудоёмкости 2: $\times, \text{ mod }, \cdot, //$

Трудоёмкость операторов цикла

Пусть для цикла вида: `for` инициализация `to` значение сравнения `do` тело

Известны трудоёмкости соответствующих блоков $f_{\text{инициализация}},$

$f_{\text{значение сравнения}}, f_{\text{тело}},$ тогда трудоёмкость цикла имеет следующий вид:

$$f = f_{\text{инициализация}} + f_{\text{значение сравнения}} + 1 + M * (f_{\text{тело}} + 1 + f_{\text{значение сравнения}} + 1) \quad (1.5)$$

Трудоёмкость условного оператора

Пусть трудоёмкость условного перехода равна 0. При известных $f_{\text{усл}}, f_{\text{тело}},$ тогда трудоёмкость цикла вида:

if усл then тело end if

Трудоёмкость равна:

$$f = f_{\text{усл}} + [0, \text{если лучший случай, иначе } f_{\text{тело}}] \quad (1.6)$$

Трудоёмкость создания массива или матрицы

Пусть трудоёмкость создания массива или матрицы равна количеству создаваемых элементов.

1.4 Вывод

Алгоритм Винограда предоставляет более оптимизированный способ умножения матриц по сравнению со стандартным методом, но его эффективность зависит от конкретной задачи и структуры данных.

2 Конструкторская часть

2.1 Требования к программному обеспечению

К разрабатываемой программе предъявлен ряд требований:

Входные данные: Две матрицы

Выходные данные: Матрица, являющаяся результатом умножения данных матриц

- Должна быть возможность одиночного выполнения умножения введенных пользователем матриц введенных пользователем 4 размеров
- Должна быть возможность массивованного замера времени выполнения реализаций алгоритмов умножения матриц
- Должна быть возможность замера процессорного времени программы
- Должна быть возможность вывода графиков и таблиц замеров процессорного времени

2.2 Разработка алгоритмов

Стандартный алгоритм умножения матриц представлен в листинге 2.1.

Листинг 2.1 Стандартный алгоритм умножения матриц

```
Input: матрицы  $A, B$   
 $rows\_A \leftarrow$  количество строк в  $A$   
 $cols\_A \leftarrow$  количество столбцов в  $A$   
 $cols\_B \leftarrow$  количество столбцов в  $B$   
Создать пустую матрицу  $result$  размером  $rows\_A \times cols\_B$ , заполненную нулями  
for  $i = 0$  to  $rows\_A - 1$  do  
    for  $j = 0$  to  $cols\_B - 1$  do  
        for  $k = 0$  to  $cols\_A - 1$  do  
             $result[i][j] = result[i][j] + A[i][k] \times B[k][j]$   
        end for  
    end for  
end for  
Return  $result$ 
```

Алгоритм умножения матриц Винограда представлен в листинге 2.2.

Листинг 2.2 Алгоритм умножения матриц Винограда

Input: матрицы A, B
 $rows_A \leftarrow$ количество строк в A
 $cols_A \leftarrow$ количество столбцов в A
 $rows_B \leftarrow$ количество строк в B
 $cols_B \leftarrow$ количество столбцов в B
Создать пустую матрицу $result$ размером $rows_A \times cols_B$, заполненную нулями
Создать массив $mulH$ размером $rows_A$, заполненный нулями
for $i = 0$ **to** $rows_A - 1$ **do**
 for $j = 0$ **to** $cols_A/2 - 1$ **do**
 $mulH[i] = mulH[i] + A[i][2 \cdot j] \times A[i][2 \cdot j + 1]$
 end for
end for
Создать массив $mulV$ размером $cols_B$, заполненный нулями
for $i = 0$ **to** $cols_B - 1$ **do**
 for $j = 0$ **to** $rows_B/2 - 1$ **do**
 $mulV[i] = mulV[i] + B[2 \cdot j][i] \times B[2 \cdot j + 1][i]$
 end for
end for
for $i = 0$ **to** $rows_A - 1$ **do**
 for $j = 0$ **to** $cols_B - 1$ **do**
 $result[i][j] = -mulH[i] - mulV[j]$
 for $k = 0$ **to** $cols_A/2 - 1$ **do**
 $result[i][j] = result[i][j] + (A[i][2 \cdot k] + B[2 \cdot k + 1][j]) \times (A[i][2 \cdot k + 1] + B[2 \cdot k][j])$
 end for
 end for
end for
if $cols_A \bmod 2 == 1$ **then**
 for $i = 0$ **to** $rows_A - 1$ **do**
 for $j = 0$ **to** $cols_B - 1$ **do**
 $result[i][j] = result[i][j] + A[i][cols_A - 1] \times B[cols_A - 1][j]$
 end for
 end for
end if
Return $result$

Оптимизированный алгоритм умножения матриц Винограда представлен в листинге 2.3. (Двоичный сдвиг вместо умножения на 2, введение декремента при вычислении вспомогательных массивов, выделение начальной итерации из

каждого внешнего цикла)

Листинг 2.3 Оптимизированный алгоритм умножения матриц Винограда

```
Input: матрицы  $A, B$   
 $rows\_A \leftarrow$  количество строк в  $A$   
 $cols\_A \leftarrow$  количество столбцов в  $A$   
 $rows\_B \leftarrow$  количество строк в  $B$   
 $cols\_B \leftarrow$  количество столбцов в  $B$   
Создать пустую матрицу  $result$  размером  $rows\_A \times cols\_B$ , заполненную нулями  
Создать массив  $mulH$  размером  $rows\_A$ , заполненный нулями  
for  $i = 0$  to  $rows\_A - 1$  do  
     $mulH[i] = -A[i][0] \times A[i][1]$   
    for  $j = 1$  to  $cols\_A/2 - 1$  do  
         $mulH[i] = mulH[i] - A[i][j \ll 1] \times A[i][(j \ll 1) + 1]$   
    end for  
end for  
Создать массив  $mulV$  размером  $cols\_B$ , заполненный нулями  
for  $i = 0$  to  $cols\_B - 1$  do  
     $mulV[i] = -B[0][i] \times B[1][i]$   
    for  $j = 1$  to  $rows\_B/2 - 1$  do  
         $mulV[i] = mulV[i] - B[j \ll 1][i] \times B[(j \ll 1) + 1][i]$   
    end for  
end for  
for  $i = 0$  to  $rows\_A - 1$  do  
    for  $j = 0$  to  $cols\_B - 1$  do  
         $result[i][j] = mulH[i] + mulV[j]$   
         $result[i][j] = result[i][j] + (A[i][0] + B[1][j]) \times (A[i][1] + B[0][j])$   
        for  $k = 1$  to  $cols\_A/2 - 1$  do  
             $result[i][j] = result[i][j] + (A[i][k \ll 1] + B[(k \ll 1) + 1][j]) \times$   
             $(A[i][(k \ll 1) + 1] + B[k \ll 1][j])$   
        end for  
    end for  
end for  
if  $cols\_A \bmod 2 == 1$  then  
    for  $i = 0$  to  $rows\_A - 1$  do  
        for  $j = 0$  to  $cols\_B - 1$  do  
             $result[i][j] = result[i][j] + A[i][cols\_A - 1] \times B[cols\_A - 1][j]$   
        end for  
    end for  
end if  
Return  $result$ 
```

2.3 Оценка трудоемкости

Пусть обрабатываются матрицы размера $M \times N$ и $N \times Q$

Стандартный алгоритм умножения матриц

Лучший и худший случай идентичны, так как алгоритм всегда выполняет одно и то же количество операций, независимо от структуры матриц.

$$\begin{aligned}
 f &= \underbrace{M \times Q}_{\text{result}} + 3 + M \times \underbrace{\underbrace{(3 + N \times \underbrace{(3 + Q \times (12 + 3))}_{\text{цикл по k}} + 3))}_{\text{цикл по j}} + 3) = \\
 &\quad \underbrace{\hspace{10em}}_{\text{цикл по i}} \\
 &= 3 + 6M + 6MN + MQ + 15MNQ
 \end{aligned} \tag{2.1}$$

Алгоритм умножения матриц Винограда

Лучший случай - $N \bmod 2 \neq 1$:

$$\begin{aligned}
 f &= \underbrace{M \times Q}_{\text{result}} + \underbrace{M}_{\text{mulH}} + 3 + M \times \underbrace{(5 + N/2 \times \underbrace{(15 + 5)}_j + 3)}_i + \\
 &+ \underbrace{Q}_{\text{mulV}} + 3 + Q \times \underbrace{(5 + N/2 \times \underbrace{(15 + 5)}_j + 3)}_i + \\
 &+ 3 + M \times \underbrace{(3 + Q \times \underbrace{(7 + 5 + N/2 \times \underbrace{(28 + 5)}_k + 3))}_{\text{цикл по j}} + 3) + 3)}_{\text{цикл по i}} + \underbrace{3}_{if} = \\
 &= 12 + 15M + 9Q + 10MN + 16MQ + 10NQ + 16.5MNQ
 \end{aligned} \tag{2.2}$$

Худший случай - $N \bmod 2 == 1$:

$$\begin{aligned}
f &= \underbrace{M \times Q}_{\text{result}} + \underbrace{M}_{\text{mulH}} + 3 + \underbrace{M \times (5 + N/2 \times (15 + 5) + 3)}_i + \\
&+ \underbrace{Q}_{\text{mulV}} + 3 + \underbrace{Q \times (5 + N/2 \times (15 + 5) + 3)}_i + \\
&+ 3 + \underbrace{M \times (3 + Q \times (7 + 5 + N/2 \times (28 + 5) + 3) + 3)}_k + \\
&\underbrace{\hspace{10em}}_{\text{цикл по j}} \\
&\underbrace{\hspace{10em}}_{\text{цикл по i}} \\
&+ 3 + 3 + \underbrace{M \times (3 + Q \times (14 + 3) + 3)}_{\text{цикл по j}} = \\
&\underbrace{\hspace{10em}}_{\text{цикл по i}} \\
&\underbrace{\hspace{10em}}_{if} \\
&= 15 + 21M + 9Q + 10MN + 33MQ + 10NQ + 16.5MNQ
\end{aligned} \tag{2.3}$$

Оптимизированный алгоритм умножения матриц Винограда

Лучший случай - $N \bmod 2 \neq 1$:

$$\begin{aligned}
f &= \underbrace{M \times Q}_{\text{result}} + \underbrace{M}_{\text{mulH}} + 3 + \underbrace{M \times (9 + 5 + (N/2 - 1) \times (13 + 5) + 3)}_i + \\
&+ \underbrace{Q}_{\text{mulV}} + 3 + \underbrace{Q \times (9 + 5 + (N/2 - 1) \times (13 + 5) + 3)}_i + \\
&+ 3 + \underbrace{M \times (3 + Q \times (6 + 18 + 5 + (N/2 - 1) \times (23 + 5) + 3) + 3)}_k + \underbrace{3}_{if} = \\
&\underbrace{\hspace{10em}}_{\text{цикл по j}} \\
&\underbrace{\hspace{10em}}_{\text{цикл по i}} \\
&= 12 + 6M + 9MN + 5MQ + 9MN + 14MNQ
\end{aligned} \tag{2.4}$$

Худший случай - $N \bmod 2 == 1$:

$$\begin{aligned}
f &= \underbrace{M \times Q}_{\text{result}} + \underbrace{M}_{\text{mulH}} + 3 + \underbrace{M \times (9 + 5 + \underbrace{(N/2 - 1) \times (13 + 5)}_j + 3)}_i + \\
&+ \underbrace{Q}_{\text{mulV}} + 3 + \underbrace{Q \times (5 + \underbrace{(N/2 - 1) \times (13 + 5)}_j + 3)}_i + \\
&+ 3 + \underbrace{M \times (3 + Q \times (6 + 18 + \underbrace{5 + \underbrace{(N/2 - 1) \times (23 + 5)}_k + 3)}_k + 3))}_{\text{цикл по j}} + \\
&\underbrace{\hspace{10em}}_{\text{цикл по i}} \\
&+ 3 + 3 + \underbrace{M \times (3 + \underbrace{Q \times (14 + 3)}_{\text{цикл по j}} + 3)}_{\text{цикл по i}} = \\
&\underbrace{\hspace{10em}}_{if} \\
&= 15 + 12M + 9MN + 22MQ + 9MN + 14MNQ
\end{aligned} \tag{2.5}$$

2.4 Вывод

В результате конструкторской части были определены требования к ПО, спроектированы стандартный алгоритм умножения матриц, алгоритм Винограда и оптимизированный по варианту алгоритм Винограда, а также произведена оценка трудоемкости алгоритмов.

3 Технологическая часть

3.1 Средства разработки

В качестве языка программирования был выбран MicroPython [1], так как он поддерживается микроконтроллером, на котором будут выполняться замеры, в его стандартной библиотеке присутствуют функция замера процессорного времени, которая требуется в условии.

Результаты замеров сохраняются в .csv файл, для построения графиков использовался exel.

Для замера времени использовалась функция `ticks_ms()` из стандартного модуля `utime` [2].

3.2 Реализация алгоритмов

В листинге 3.1 представлена реализация стандартного алгоритма умножения матриц.

Листинг 3.1 — Стандартный алгоритм умножения матриц

```
def Multiply(A, B):
    rows_A = len(A)
    cols_A = len(A[0])
    rows_B = len(B)
    cols_B = len(B[0])

    result = [[0 for _ in range(cols_B)] for _ in range(rows_A)]

    for i in range(rows_A):
        for j in range(cols_B):
            for k in range(cols_A):
                result[i][j] += A[i][k] * B[k][j]

    return result
```

В листинге 3.2 представлена реализация алгоритма умножения матриц Винограда.

Листинг 3.2 — Алгоритм умножения матриц Винограда

```
def VinogradMultiply(A, B):
    rows_A = len(A)
    cols_A = len(A[0])
    rows_B = len(B)
    cols_B = len(B[0])

    result = [[0 for _ in range(cols_B)] for _ in range(rows_A)]

    mulH = [0] * rows_A
    for i in range(rows_A):
        for j in range(0, cols_A // 2):
            mulH[i] = mulH[i] + A[i][2 * j] * A[i][2 * j + 1]

    mulV = [0] * cols_B
    for i in range(cols_B):
        for j in range(0, rows_B // 2):
            mulV[i] = mulV[i] + B[2 * j][i] * B[2 * j + 1][i]

    for i in range(rows_A):
        for j in range(cols_B):
            result[i][j] = -mulH[i] - mulV[j]
            for k in range(0, cols_A // 2):
                result[i][j] = result[i][j] + (A[i][2 * k] + B[2 * k
                    + 1][j]) * (A[i][2 * k + 1] + B[2 * k][j])

    if cols_A % 2 == 1:
        for i in range(rows_A):
            for j in range(cols_B):
                result[i][j] = result[i][j] + A[i][cols_A - 1] * B[
                    cols_A - 1][j]

    return result
```

В листинге 3.3 представлена реализация оптимизированного алгоритма умножения матриц Винограда.

- двоичный сдвиг вместо умножения на 2
- введение декремента при вычислении вспомогательных массивов
- вынос начальной итерации из каждого внешнего цикла

Листинг 3.3 — Алгоритм умножения матриц Винограда

```
def OptimVinogradMultiply(A, B):
    rows_A = len(A)
    cols_A = len(A[0])
    rows_B = len(B)
    cols_B = len(B[0])

    result = [[0 for _ in range(cols_B)] for _ in range(rows_A)]
    mulH = [0] * rows_A
    for i in range(rows_A):
        mulH[i] -= A[i][0] * A[i][1]
        for j in range(1, cols_A // 2):
            mulH[i] -= A[i][j << 1] * A[i][(j << 1) + 1]
    mulV = [0] * cols_B
    for i in range(cols_B):
        mulV[i] -= B[0][i] * B[1][i]
        for j in range(1, rows_B // 2):
            mulV[i] -= B[j << 1][i] * B[(j << 1) + 1][i]
    for i in range(rows_A):
        for j in range(cols_B):
            result[i][j] = mulH[i] + mulV[j]
            result[i][j] = result[i][j] + (A[i][0] + B[1][j]) * (A[i][1] + B[0][j])
            for k in range(1, cols_A // 2):
                result[i][j] = result[i][j] + (A[i][k << 1] + B[(k << 1) + 1][j]) * (A[i][(k << 1) + 1] + B[k << 1][j])
    if cols_A % 2 == 1:
        for i in range(rows_A):
            for j in range(cols_B):
                result[i][j] = result[i][j] + A[i][cols_A - 1] * B[cols_A - 1][j]
    return result
```


3.3 Функциональные тесты

В табличке 3.1 приведены функциональные тесты для алгоритмов.

№	Входные данные	Результат	Ожидаемое
1	$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$ $B = \begin{pmatrix} 9 & 8 & 7 \\ 6 & 5 & 4 \\ 3 & 2 & 1 \end{pmatrix}$	$C = \begin{pmatrix} 30 & 24 & 18 \\ 84 & 69 & 54 \\ 138 & 114 & 90 \end{pmatrix}$	Результат совпадает с рассчитанной матрицей С.
2	$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$ $B = \begin{pmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{pmatrix}$	$C = \begin{pmatrix} 58 & 64 \\ 139 & 154 \end{pmatrix}$	Матрица С совпадает с ожидаемой.
3	$A = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 0 & 1 & 2 & 3 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 1 \end{pmatrix}$ $I = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$	$A = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 0 & 1 & 2 & 3 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 1 \end{pmatrix}$	Результат совпадает с исходной матрицей А.

Таблица 3.1 — Функциональные тесты для алгоритма умножения матриц с конкретными данными

Все тесты пройдены успешно

3.4 Вывод

В ходе работы были разработаны стандартный алгоритм и алгоритм Винограда умножения матриц и оптимизированный по варианту алгоритм Винограда, а также проведено их тестирование.

4 Исследовательская часть

4.1 Технические характеристики

Технические характеристики устройства, на котором проводились замеры:

- ARM Cortex-M7 core 216 МГц
- Оперативная память: 2 Мб

4.2 Временные характеристики

В таблице 4.1 приведены временные характеристики, полученные в результате замеров времени алгоритмов. Для каждого размера квадратной матрицы замер проводился 100 раз, при этом для каждой итерации создавались 2 случайные матрицы заданного размера и использовались как аргументы к алгоритмам. Итоговое время взято как среднее арифметическое всех полученных замеров.

Размер матрицы, $N \times N$	Стандартный, нс	Виноград, нс	Виноград оптимизированный, нс
15	2.5	2.71	2.54
20	5.86	6.11	5.73
25	11.33	11.74	11.12
30	19.45	19.86	18.85
35	31.01	31.26	29.93

Таблица 4.1 — Сравнение времени выполнения для различных методов умножения матриц

Полученные замеры также можно увидеть на графике 4.1:

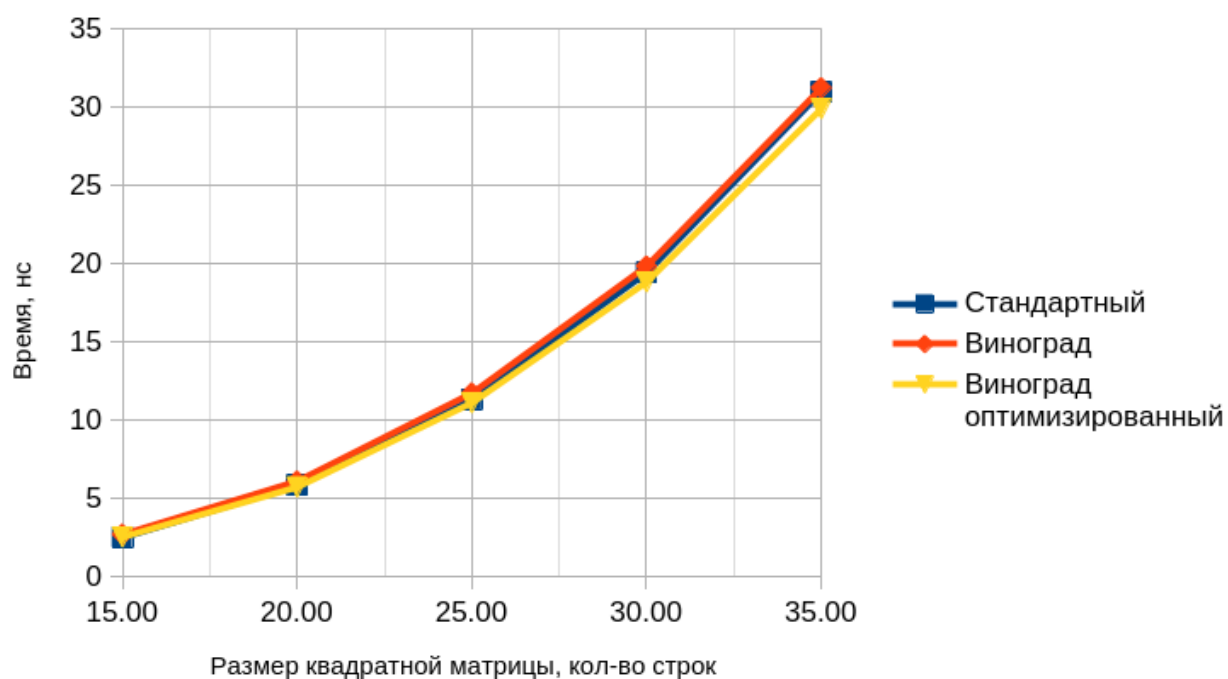


Рисунок 4.1 — График зависимости времени работы алгоритмов от размера матриц

Как видно из таблицы 4.1 и графика 4.1, оптимизированный алгоритм оказался более эффективным.

4.3 Вывод

Из исследования видно, что неоптимизированный алгоритм Винограда уступает в эффективности стандартному, однако при его оптимизации получается получить выигрыш по времени.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы была достигнута цель освоения методов оценки трудоемкости алгоритмов умножения матриц. Алгоритм Винограда показал значительное преимущество в производительности по сравнению со стандартным методом, особенно при увеличении размеров матриц. Проведённые замеры и графический анализ подтвердили эффективность оптимизации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. MicroPython, Python compiler and runtime that runs on the bare-metal. / [Электронный ресурс] // MicroPython : [сайт]. — URL: <https://micropython.org/> (дата обращения: 14.10.2024).
2. MicroPython-utime, time related functions / [Электронный ресурс] // MicroPython-utime : [сайт]. — URL: <https://docs.micropython.org/en/v1.15/library/utime.html> (дата обращения: 14.10.2024).
3. Корн Г., Корн Т. Алгебра матриц и матричное исчисление // Справочник по математике. — 4-е издание. — М.: Наука, 1978. — С. 392—394.
4. Henry Cohn, Robert Kleinberg, Balazs Szegedy, and Chris Umans. Group-theoretic Algorithms for Matrix Multiplication. arXiv:math.GR/0511460. Proceedings of the 46th Annual Symposium on Foundations of Computer Science, 23-25 October 2005, Pittsburgh, PA, IEEE Computer Society, pp. 379—388.